W5 IIDB262-THU

1. Flex: 1차원 컴포넌트 레이아웃
2. Flexbox와 Grid 비교
3. Grid: 2차원 골조 및 격자 레이아웃
4. 추가자료
5. 과제
    1. CSS Grid Garden
    2. 플렉스, 그리드 사용한 레이아웃 구현
        [Part 1. Flexbox 실습 (7개)]
        [Part 2. Grid 실습 (8개)]
    3. 제출 방식

SMU-IIDB - a Collection by spctrm404 on CodePen
https://codepen.io/collection/xeYdGb

본 주차의 목적은 CSS의 핵심 레이아웃 모듈인 Flexbox(1차원)와 Grid(2차원)의 설계 철학 차이를 이해하고, 실무 웹 UI 환경에서 각각 어떤 상황에 사용해야 하는지(Use Case)를 판단하는 기준을 확립하는 것입니다.

1. Flex: 1차원 컴포넌트 레이아웃

Flexbox는 단방향(가로 혹은 세로 중 하나의 축)으로 요소를 배치하고 정렬하는 데 특화되어 있습니다. 부모 영역을 미리 나누기보다는, 내부 콘텐츠의 크기에 따라 유연하게 공간을 분배(Content-out)하는 컴포넌트 단위의 레이아웃에 주로 사용됩니다.

[대표적인 Use Case]

- **네비게이션 바 (GNB/LNB):** 좌측 로고, 중앙 메뉴, 우측 버튼 그룹 등 요소들을 양끝으로 밀어내거나 중앙에 정렬할 때 (`justify-content: space-between`, `align-items: center`).
- **검색바 및 입력 폼:** 가변적인 너비를 가지는 텍스트 입력창과 고정 너비를 가지는 검색 버튼을 한 줄에 꽉 차게 배치할 때 (`flex-grow`).

- **태그(Tag) / 칩(Chip) 래핑:** 텍스트 길이에 따라 너비가 불규칙한 요소들을 한 줄로 나열하다가, 부모의 너비를 초과하면 자연스럽게 다음 줄로 넘길 때 (`flex-wrap: wrap`).
- **미디어 오브젝트:** 썸네일 이미지(고정)와 텍스트 본문(가변)을 나란히 배치하고 수직 중앙을 맞출 때.

💡 **[자습용 검색 키워드]**

- `CSS Flexbox flex-grow, flex-shrink, flex-basis`
- `CSS Flexbox justify-content vs align-items`
- `CSS Flexbox wrap`

2. Flexbox와 Grid 비교

Flexbox와 Grid는 상호 보완적인 도구입니다. 요구사항에 따라 더 효율적인 레이아웃 모델을 선택해야 합니다.

[Grid로는 쉬운데 Flexbox로는 어려운 것]

- **2차원(행과 열) 동시 통제:** Flexbox에서 줄 바꿈(`wrap`)이 발생하면 윗줄과 아랫줄 요소의 세로선(열)을 일치시키기 어렵습니다. Grid는 처음부터 격자를 정의하므로 요소의 높낮이나 너비가 달라도 열과 행이 정확히 맞습니다.
- **요소의 다중 칸 차지 (Spanning):** 특정 요소가 2열 2행을 동시에 차지하는 복잡한 구조(예: 벤토박스)를 만들 때, Flexbox는 HTML 부모 컨테이너를 여러 번 중첩해야 하지만 Grid는 CSS 속성만으로 병합이 가능합니다.
- **요소 겹치기 (Overlapping):** `position: absolute`를 사용하지 않고도, Grid의 동일한 셀(Cell)을 지정하여 배경과 텍스트를 손쉽게 겹칠 수 있습니다.

[Flexbox로는 쉬운데 Grid로는 어려운 것]

- **윗줄과 아랫줄의 완벽한 독립 (탈격자 흐름):**
  Grid는 다음 줄로 줄바꿈이 일어날 때, 아랫줄 요소가 윗줄 요소의 세로선(Column) 격자에 강제로 묶이게 됩니다. 반면 Flexbox(`flex-wrap: wrap`)는 윗줄과 아랫줄이 서로의 너비에 전혀 영향을 주지 않습니다. 요소들이 각자의 글자 길이(내용물)만큼만 공간을 차지하며 물 흐르듯 자연스럽게 떨어지는 구조(예: 해시태그 리스트, 칩 레이아웃)에는 Flexbox가 압도적으로 유리합니다.
- **부모 개입 없는 자식 주도적 공간 배치:**
  Grid는 배치 전 '부모 요소'가 미리 자식의 개수와 열의 너비를 정확히 계산해서 칸을 쪼개두어야 하는 제약이 있습니다. 반면 Flexbox는 부모의 복잡한 계산 없이 자식 요소 스스로 배치를 결정할 수 있습니다. 예를 들어 네비게이션 메뉴의 개수가 가변적으로 변하

더라도, 우측 버튼 요소에 `margin-left: auto` 속성 하나만 주면 스스로 남는 공간을 모두 밀어내어 우측 끝에 배치됩니다.

3. Grid: 2차원 골조 및 격자 레이아웃

Grid는 **행(Row)과 열(Column)을 동시에 제어**하는 2차원 레이아웃 시스템입니다. 부모 컨테이너가 먼저 명확한 격자를 설계하고, 자식 요소들을 해당 격자에 배치하는 **레이아웃 중심(Layout-in)** 방식입니다.

[대표적인 Use Case]

- **반응형 카드 갤러리 (Auto-fit):** 미디어 쿼리(Media Query) 없이, 브라우저 너비에 따라 카드가 4열, 3열, 2열로 자동 변경되는 상품 리스트 뷰 (`repeat(auto-fit, minmax(...))`).
- **벤토박스 (Bento Box) 및 대시보드:** 다양한 크기의 위젯들이 1x1, 2×1, 2×2 등의 면적을 차지하며 빈틈없이 맞물려 있는 UI (`grid-column`, `grid-row`).
- **웹앱 셸 (App Shell):** 좌측 사이드바, 상단 헤더, 메인 콘텐츠 영역으로 이루어진 슬랙/노션 형태의 전체 화면 골조 설계.
- **복잡한 결제/입력 폼:** [성] [이름] 2칸, [주소] 1칸 전체, [국가] [도시] [우편번호] 3칸 등 열의 개수가 줄마다 바뀌는 정교한 폼 레이아웃.
- **에디토리얼 / 포스터 레이아웃 (레이어 겹침 및 비대칭 정렬):** `position: absolute`를 사용하지 않고, 부모 Grid의 동일한 셀(`grid-area: 1 / 1`)을 지정하여 배경 이미지와 텍스트 레이어를 우아하게 겹치는 기법입니다. 텍스트 레이어 내부에 다시 격자를 선언한 뒤 각 텍스트 블록을 특정 열(Column)과 행(Row)의 끝단(`align-self: end`)에 분산 배치하면, 스위스 모던 타이포그래피(요세프 뮬러 브로크만 스타일)처럼 엄격한 비대칭 격자 기반의 정교한 시각적 계층 구조를 손쉽게 구현할 수 있습니다.

💡 **[자습용 검색 키워드]**

- `CSS Grid grid-template-columns, grid-template-rows`
- `CSS Grid grid-column, grid-row spanning`
- `CSS Grid repeat, auto-fit, minmax`

4. 추가자료

이번에야말로 CSS Flex를 익혀보자
스튜디오밀(1분코딩) — 인터랙티브 웹 개발 튜토리얼
https://studiomeal.com/archives/197/

이번에야말로 CSS Grid를 익혀보자
스튜디오밀(1분코딩) — 인터랙티브 웹 개발 튜토리얼
https://studiomeal.com/archives/533/

https://youtu.be/wsTv9y931o8?si=7uQHiayH7K48JW1k

https://youtu.be/JYfiaSKeYhE?si=-glTa5×5dXByewqV

5. 과제

1. CSS Grid Garden

클리어 스크린샷

Grid Garden
A game for learning CSS grid layout
https://cssgridgarden.com/#ko



2. 플렉스, 그리드 사용한 레이아웃 구현

디자인 속성(색상, 꾸미기)을 배제하고, CSS Flexbox와 Grid를 이용한 공간 배치(Layout) 능력만을 반복 훈련하여 두 모델의 차이와 적절한 사용처를 체화합니다.

- **꾸미기 금지:** 모든 요소는 `border`와 가벼운 `background-color` 정도만 부여하여 박스의 크기와 위치만 식별하게 합니다.
- **반응형 크기 적용:** 고정된 `width`나 `height` 픽셀값 사용을 지양하고, 내용물에 따라 늘어나거나 부모 너비에 맞춰 유연하게 반응하는 구조로 작성하십시오.

[Part 1. Flexbox 실습 (7개)]

1. **GNB (글로벌 네비게이션):** 좌측 로고, 우측 로그인 버튼 그룹 양끝 정렬
2. **검색바 (Search Bar):** 가변 너비의 입력창과 고정 너비의 검색 아이콘을 한 줄에 꽉 차게 배치
3. **미디어/프로필 객체:** 좌측 원형 썸네일과 우측 텍스트 정보의 수직 중앙 정렬
4. **카드 내 액션 버튼 하단 고정:** 카드 본문 길이에 관계없이 '자세히 보기' 버튼은 카드 최하단에 항상 위치
5. **해시태그 / 필터 칩(Chip):** 여러 개의 태그가 한 줄로 나열되다 부모 너비를 초과하면 다음 줄로 자동 줄바꿈
6. **동등 비율 탭(Tab) 메뉴:** 너비와 상관없이 3개의 탭 메뉴가 1:1:1의 동일 비율로 배분
7. **모바일 하단 툴바:** 앱 하단에 4개의 아이콘이 동일한 여백 간격을 두고 고르게 배치

[Part 2. Grid 실습 (8개)]

8. **커머스 상품 리스트 (Auto-fit):** 카드의 최소 너비 제한을 두고, 화면 넓이에 따라 열의 개수가 자동으로 반응하는 갤러리 뷰
9. **대시보드 / 벤토박스:** 다수의 위젯 중 특정 위젯 1개만 가로/세로 2칸(2x2)을 병합하여 차지하는 구조
10. **복잡한 결제 폼:** 첫 줄 2칸, 둘째 줄 1칸, 셋째 줄 3칸으로 분할된 정교한 회원가입 폼 레이아웃
11. **월간 달력 (Calendar):** 7개의 열(Column)을 지정하여 자동으로 날짜가 7칸 격자에 맞추어 떨어지는 레이아웃
12. **스펙 비교 표 (Pricing Table):** `<table>` 태그 없이 Grid를 사용하여 각 행과 열의 크기가 일관되게 정렬된 요금표
13. **반응형 메가 푸터:** 데스크탑 4열, 태블릿 2열, 모바일 1열로 변환되는 사이트 하단 정보 영역
14. **이미지 위 텍스트 겹침 (Hero Banner):** `position: absolute`를 사용하지 않고 Grid 셀(Cell) 병합을 통해 이미지 위에 텍스트 박스를 오버랩(겹침) 배치
15. **웹앱 셸 (App Shell):** 화면 전체 높이(`100vh`)를 기준으로 좌측 사이드바, 상단 헤더, 메인 콘텐츠 영역의 전체 화면 뼈대 설계

3. 제출 방식

- **제출 기한:** 다음 수업 시작 전까지

- **제출 방법:** 클리어 스크린샷 피그마 url과 CodePen의 **Flexbox 7종 링크 1개**와 **Grid 8종 링크 1개**(총 2개의 URL)를 분리하여 제출합니다. (스크롤하며 확인할 수 있도록 각 컴포넌트 간 상하 여백을 추가하십시오.)
- **제출 링크:**

W5 IIDB262-THU

https://forms.gle/frAzzdEFt2scPsL27

